

SIMPLIFIED PRIORS FOR OBJECT-CENTRIC LEARNING

Vihang Patil^{*1} Andreas Radler^{*1} Daniel Klotz² Sepp Hochreiter^{1,3}

^{*}Equal contribution

¹LIT AI Lab, Institute for Machine Learning, JKU Linz, Austria

²Helmholtz Centre for Environmental Research (UFZ), Leipzig, Germany

³NXAI GmbH, Linz, Austria

{patil, radler, klotz, hochreit}@ml.jku.at

ABSTRACT

Humans excel at abstracting data and constructing *reusable* concepts, a capability lacking in current continual learning systems. The field of object-centric learning addresses this by developing abstract representations, or slots, from data without human supervision. Different methods have been proposed to tackle this task for images, whereas most are overly complex, non-differentiable, or poorly scalable. In this paper, we introduce a conceptually simple, fully-differentiable, non-iterative, and scalable method called **SAMP** (Simplified Slot Attention with **Max Pool Priors**). It is implementable using only Convolution and MaxPool layers and an Attention layer. Our method encodes the input image with a Convolutional Neural Network and then uses a branch of alternating Convolution and MaxPool layers to create specialized sub-networks and extract primitive slots. These primitive slots are then used as queries for a Simplified Slot Attention over the encoded image. Despite its simplicity, our method is competitive or outperforms previous methods on standard benchmarks.

1 INTRODUCTION

In a dynamic environment, it is essential for artificial intelligence systems to be able to effectively adapt and generalize to similar unseen input. The field of continual learning aims to design adaptable systems, while mitigating forgetting of important previously acquired capabilities (i.e. catastrophic forgetting). Many current methods approach continual learning via memory stability, or inter-task generalizability mechanisms (Wang et al., 2024), while being agnostic to the underlying representation. Recently object-centric learning has gained momentum as a promising direction to build learning systems inspired by human cognition (Locatello et al., 2020; Greff et al., 2019; Burgess et al., 2019; Greff et al., 2020). The core idea is, that an object-centric representation of the raw-input may lead to better generalization and adaptation capabilities. These symbolic-like representations for objects, can then be applied, reused, and combined for reasoning or decision-making (Spelke & Kinzler, 2007). However, despite the rise of Deep Learning (Krizhevsky et al., 2012; Schmidhuber, 2015) and access to large scale compute and data (Deng et al., 2009) that led to human level performance in various domains (He et al., 2016; Mnih et al., 2015; Silver et al., 2016), neural networks do not possess this abstraction ability.

Various recent works have tried to address the problem of abstraction from raw input by learning object-centric representations — often known as slots (Locatello et al., 2020; Wu et al., 2023; Chang et al., 2023; Engelcke et al., 2019; Burgess et al., 2019) — from high-dimensional input, such as images, videos, 3D point clouds or even 3D scenes (images of different angles onto the same objects). However, most of the currently proposed methods are complex and partially suffer from training instabilities (Chang et al., 2023). Many of these methods are derived from the original Slot Attention idea (Locatello et al., 2020), which applies a modification of Cross-Attention (Vaswani et al., 2017) in an iterative refinement procedure. We argue that such an iterative refinement procedure is not desired and propose a solution that can serve as a simple baseline.

Prior to explicit slot extraction methods, *competition mechanisms* (Srivastava et al., 2013) like MaxPooling or Spatial Pyramid MaxPooling (He et al., 2014) showed a strong performance (He et al., 2015; 2017; Szegedy et al., 2014) to obtain highly relevant abstract features. Srivastava et al. (2014b) empirically showed that competition among subnetworks leads to specialization, whereas for different input, different subnetworks are activated. We argue that this ability to specialize and abstract are useful properties for learning object-centric representations.

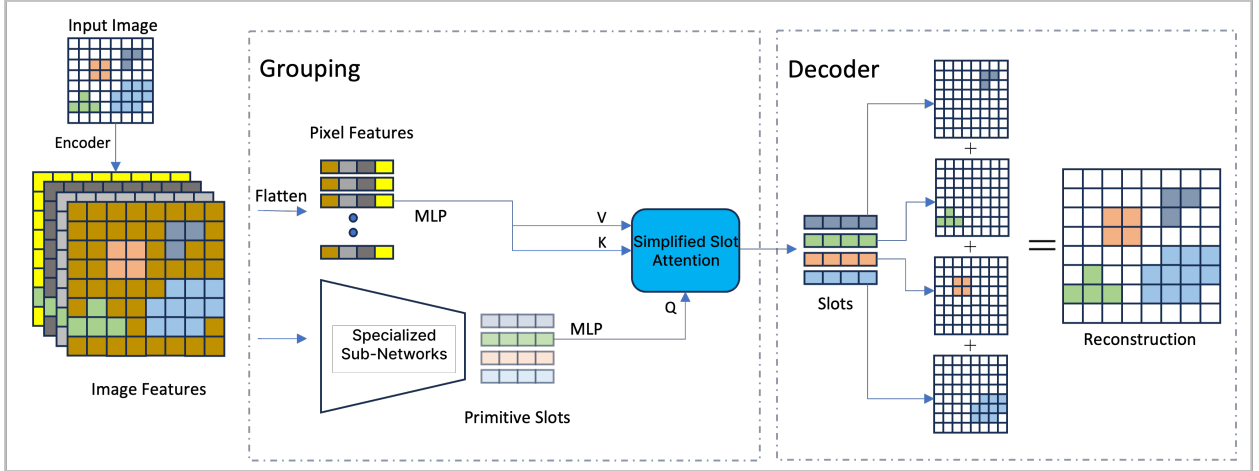


Figure 1: *Grouping*: We learn Primitive Slots from image features using Specialized Sub-Networks. We obtain pixel features by flattening all the image features from the encoder. We pass pixel features and Primitive Slots to a Simplified Slot Attention (SSA) layer, where *Keys* (K) and *Values* (V) are the pixel features and *Queries* are the Primitive Slots. SSA layer outputs the slots. The decoder is applied on every slot separately to reconstruct the input and a mask. A softmax is applied to the masks along the pixel dimension (for simplicity the masks are not shown in the figure). The final reconstruction is obtained by performing a weighted sum of all the individual reconstructions across the pixels with the weights coming from the masks.

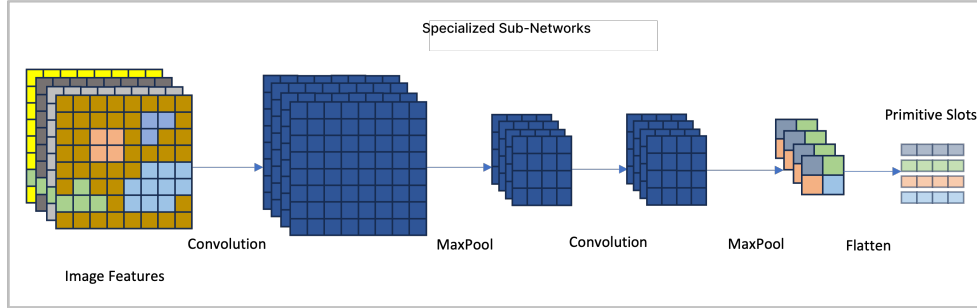


Figure 2: *Specialized Sub-Networks*: We use alternating Convolution and MaxPool layers. After these layers, we flatten features to obtain Primitive Slots. The architecture along with the slot-wise reconstruction in the decoder, induces specialization in sub-networks. The sub-networks are forced to explain different parts of the input. Therefore, the resultant Primitive Slots are good queries for the SSA layer.

In this paper, we present a novel method that we call Simplified Slot Attention with Max Pool Priors (SAMP). SAMP is a simple, scalable, non-iterative and fully-differentiable approach to extract slots from images with simple Convolutions (CNN; [LeCun et al., 2004](#)) and MaxPool blocks ([Krizhevsky et al., 2012](#)) that are combined with a Simplified Slot attention (SSA) layer. The effectiveness of SAMP demonstrates that the iterative nature of Slot Attention based methods is not necessary.

Our main contributions are:

- We propose SAMP (Simplified Slot Attention with Max Pool Priors) a method for learning object-centric representations.
- SAMP is a simple, but scalable baseline for OCL, since it is non-iterative and consists of vanilla building blocks like CNN, MaxPool layers and a Simplified Slot-Attention.
- We evaluate SAMP on standard Object-Centric benchmarks, where it is competitive or outperforms various other Object-Centric methods.

Algorithm 1 Simplified Slot Attention (SSA) Layer: SSA removes the iteration from Slot Attention, along with GRU and the MLP layers. It also has shared projection weights for keys and values. Further, we do not normalize across keys after the softmax over slots.

```

1: Input: inputs  $\in \mathbb{R}^{P \times D}$ , slot_priors  $\in \mathbb{R}^{n \times D}$ 
2: Layer Params: k (shared projection for keys and values), q; projection for slot priors; LayerNorm(x2)
3: inputs = LayerNorm(inputs)
4: slots = LayerNorm(slot_priors)
5:  $M = \frac{1}{\sqrt{n}} k(\text{inputs}) \cdot q(\text{slots})^T$ 
6:  $W_{i,j} = \frac{e^{M_{i,j}}}{\sum_{l=1}^n e^{M_{l,j}}}$ 
7: slots =  $W^T \cdot k(\text{inputs})$ 
8: return slots

```

Algorithm 2 Slot Attention Module as given in [Locatello et al. \(2020\)](#)

```

1: Input: inputs  $\in \mathbb{R}^{P \times D}$ , slots  $\sim \mathcal{N}(\mu, \text{diag}(\sigma)) \in \mathbb{R}^{n \times D}$ 
2: Layer Params: k, q, v: linear projections for attention; GRU; MLP; LayerNorm(x3)
3: inputs = LayerNorm(inputs)
4: for  $t = 0 \dots T$  do
5:   slots_prev = slots
6:   slots = LayerNorm(slots)
7:    $M = \frac{1}{\sqrt{D}} k(\text{inputs}) \cdot q(\text{slots})^T$ 
8:    $\text{attn}_{i,j} = \frac{e^{M_{i,j}}}{\sum_{l=1}^n e^{M_{l,j}}}$ 
9:    $W_{i,j} = \frac{\text{attn}_{i,j}}{\sum_{l=1}^P \text{attn}_{l,j}}$ 
10:  updates =  $W^T \cdot v(\text{inputs})$ 
11:  slots = GRU(state=slots_prev, inputs=updates)
12:  slots += MLP(LayerNorm(slots))
13: end for
14: return slots

```

2 RELATED WORK

Object-centric learning is related to abstraction, which is a well studied topic in different areas of Machine Learning ([Givan et al., 2003](#); [Ravindran & Barto, 2003](#); [Sutton et al., 1999](#); [Li et al., 2006](#); [Vezhnevets et al., 2017](#); [Kulkarni et al., 2016](#); [Patil et al., 2022](#); [2023](#)). Object-centric learning methods try to extract representations of objects given an input, which could be images or some other modality. These extracted representations are known as slots.

Many such methods consist of three modules: an image encoder, a grouping module and a decoder. The image encoders are typically realized with multiple CNN layers ([Locatello et al., 2020](#); [Engelcke et al., 2021](#)). For the decoder, the spatial broadcast decoder ([Watters et al., 2019](#)) — or slight deviations thereof — is the de-facto standard, since it provides a good inductive bias for learning disentangled representations ([Engelcke et al., 2020](#)). The main difference between common slot extraction methods is the *grouping module*. Most grouping modules can be divided into (a) graph-based, (b) generative and (c) iterative refinement based approaches.

Graph-based approaches [Pervez et al. \(2022\)](#) use a graph-cutting approach to perform clustering of pixel-features via a quadratic program. The disadvantage of this approach is that solving the underlying quadratic program is computationally intense and poorly parallelizable.

Generative approaches aim to learn parameterized distributions of latent variables by minimizing a proxy of the log-likelihood. Works such as IODINE ([Greff et al., 2019](#)), MONet ([Burgess et al., 2019](#)), SPACE ([Lin et al., 2020](#)) or Genesis ([Engelcke et al., 2019](#)) fall into this category. While these methods are theoretically grounded, they lack empirical performance. Genesis-V2 [Engelcke et al. \(2021\)](#) is an extension of Genesis and one of the first fully-differentiable methods that is able to choose the number of slots *by design* during training and inference (i.e., not by doing a second forward pass with a different number of slots). Although this selection option brings some interesting properties, the method has not found wide adoption. We argue that the reason for this is that it is complex in terms of math and code.

Iterative refinement approaches The Slot Attention module ([Locatello et al., 2020](#)) — often simply called Slot Attention — is a successful and widely adapted technique. Thus, there exists a sizable body of work that is dedicated to improve it: Implicit Slot Attention (ISA [Chang et al., 2023](#)) stabilizes the iterative refinement process of Slot Attention by learning the fixed-point of the iterative refinement procedure with a first-order Neumann series approximation. While it stabilizes the training, it tends to need even more iterations than Slot Attention which increases the overall time complexity. [Jia et al. \(2023\)](#) extend Implicit Slot Attention by learning distributions for initializations of the iterative refinement procedure. An other improvement suggestion to the original Slot Attention module came from ([Kim et al., 2023](#)) who introduced a locality-bias as it is common for CNNs in the attention part of Slot Attention. Similarly, [Gao et al. \(2023\)](#) improved the slot initialization with explicit clustering methods, such as Mean-Shift ([Comaniciu & Meer, 2002](#)) or K-means ([Lloyd, 1982](#)). SLATE ([Singh et al., 2022](#)) uses Slot Attention as the grouping module, but replaces

CNN encoder with a discrete Variational Autoencoder (Ramesh et al., 2021) encoder and the CNN spatial broadcast decoder (Watters et al., 2019) with an autoregressive transformer decoder (Vaswani et al., 2017).

Competition in Neural Networks Competition in artificial neural networks is inspired by observations of the human brain. Competitive dynamics among neurons and neural circuits have played a crucial role in understanding brain processes in biological models Eccles et al. (1967); Ermentrout (1992). The significance stems from early findings that revealed a recurring "on-center, off-surround" neural architecture in various brain regions Ellias & Grossberg (1975). The "on-center, off-surround" architecture involves neurons providing excitatory feedback locally and inhibitory signals broadly. Therefore it creates competition between sub-networks (Andersen et al., 1969; Ellias & Grossberg, 1975). The biological model in Lee et al. (1999) is also of interest, as it proposes that attention activates a winner-take-all competition over the visual features in the human brain. Inspired from the brain, such competitive networks have made their way to machine learning (Maass, 1999; 2000). Hamming Networks (Lippmann, 1987) perform iterative lateral inhibition between competitors. Local-Winner-Take-All networks (Srivastava et al., 2014a) setup blocks of neurons, and pass the winning activation of each block to the next layer. Adding sources of competition on networks trained on faces, showed impressive abstraction capabilities, where feature maps of deeper layers (i.e., closer to the output) become more abstract and depict parts of a face (Wang & Tan, 2014).

3 METHOD

We propose *Simplified Slot Attention with Max Pool Priors*, a method to learn Object-Centric representations from images. Most architectures consist of three main modules: an encoder, a grouping and a decoder module. The encoder computes useful features from the raw image, the grouping module extracts object-representations (slots) and the decoder reconstructs the individual slots to partial images, which are then summed up to obtain the input image in an auto-encoder fashion. The main novelty of SAMP lies in its simplified grouping mechanism (Figure 1 and 2). SAMP groups image features using specialized sub-networks and a variant of the slot attention layer. Competition and specialization is induced in SAMP due to the following components: 1) Winner Take All MaxPool layers 2) Simplified Slot Attention layer 3) Spatial Broadcast Decoder.

In the following sub-section (3.1), we first go through the architecture of SAMP. Afterwards, we describe the sources of competition and specialization in SAMP (sub-section 3.2).

3.1 ARCHITECTURE

The SAMP architecture consists of an encoder, grouping module and a decoder.

Encoder The encoder is a simple image encoder with CNN layers (LeCun et al., 2004). We have a fixed kernel size, padding and stride across these layers. The layers are setup in such a way that the spatial dimension of the input is preserved. For example, if the input image is of size $H \times W$ then the encoder output is $H \times W \times c$, where, H and W are the height and width of the image. c is the number of filters used in the last layer. This is done to obtain features at pixel level, which can then be later grouped together to obtain slots. Similar to Locatello et al. (2020), we also augment the pixel features with a positional embedding. For more details, see Appendix A.2.

Grouping The output of the encoder is fed to SAMP’s grouping module, which outputs slots. The grouping module consists of specialized sub-networks and a SSA Layer.

The specialized sub-networks ingest the encoder output and return primitive-slots. A specialized sub-networks consist of MaxPool and Convolution layers. Similar to the encoder, we build the CNN layers in such a way that they preserve the spatial dimension. But, the MaxPool layers reduce it (Figure 2). After applying CNN and MaxPool layers we obtain an output of size $n_h \times n_w \times c$. Here, n_h, n_w are height and width of the output and c is the number of filters of the last CNN layer. We flatten the output to obtain, $n \times c$, where $n = n_h \times n_w$ are the number of primitive slots. Conceptually, n is therefore a hyperparameter of the network. We pass these primitive-slots through an MLP layer and then use them as queries in the SSA Layer.

The SSA Layer is a variant of the standard slot attention layer (Locatello et al., 2020). We remove the multiple iterations done in slot attention along with the Gated recurrent unit (GRU) layer (Chung et al., 2014) and Multilayer perceptron (MLP) layer. We also share projection weights between the keys and values, thus, keys and values are same in our architecture. Unlike, slot attention, after the softmax over the queries, we do not normalize attention coefficients across the keys. The softmax over the queries creates competition in the slots to explain different parts of the input. In our case, as the keys and values share the same projection weights it can also be thought of as an associative memory.

In that case it is a storage of patterns (Ramsauer et al., 2020; Widrich et al., 2021; Paischer et al., 2022), from which we retrieve patterns (values) similar to primitive slots (queries).

SAMP first flattens the encoder output and passes it through MLPs so they can be used as keys ($\mathbf{K} \in \mathbb{R}^{P \times D}$) and values ($\mathbf{V} \in \mathbb{R}^{P \times D}$) in the SSA Layer layer, whereas P is the number of pixels and D is the dimensionality of the slots. The primitive-slots are used as queries ($\mathbf{Q} \in \mathbb{R}^{n \times D}$). Once we get the attention coefficients, we can use them to get the slots ($\mathbf{S} \in \mathbb{R}^{n \times D}$) as follows by using

$$\mathbf{S} = \mathbf{W}^T \mathbf{V} \quad \text{where, } \mathbf{W} = \text{softmax}\left(\frac{\mathbf{K}\mathbf{Q}^T}{\tau}\right) \quad (1)$$

The temperature τ is set to $\sqrt{n}$, whereas n is the number of slots. This is a lower temperature then commonly used in Cross-Attention (Vaswani et al., 2017). Empirically we found that the resulting lower-entropy distribution of attention values leads to better results.

Decoder The decoding treats each slot separately with a Spatial Broadcast Decoder (Greff et al., 2019). Every slot reconstructs its own image and a mask. The masks are normalized and a softmax along the pixel dimension of all masks mixes all the images together. We train the model end to end with a mean squared error reconstruction loss (similar to Greff et al., 2019; Locatello et al., 2020).

Our method is able to extract slots, without any iterative refinement. In the next subsection, we explain how SAMP creates competition amongst the slots to explain different parts of the input.

3.2 COMPETITION IN SAMP

SAMP creates competition and specialization through different sources: The MaxPool Layers, the SSA Layer layer and a slot wise reconstruction decoder.

Specialization through MaxPool layers MaxPool layers are interleaved between Convolution layers (Figure 2). MaxPool layers work by transmitting activations of winning units to the next layer. As there is only one winner for a local group of neurons, the layer activations are sub-sampled. This results in sub-networks competing to have higher activations. During back-propagation, units that win and the subnetworks that are responsible for this will get updated. As a result, a winning sub-network is reinforced to win more if it predicts correctly. If a sub-network wins, and does not predict correctly, it is not reinforced. One can also look at this as a gradient-based search over finding sub-networks, which explain the input correctly. An important side effect of these specialized sub-networks is that the resulting primitive-slots explain different parts of the input. A neuron has a higher chance of winning if it explains a different part of the input, rather than explaining the same feature as another neuron.

Competition through SSA Layer Another source of competition for SAMP is the SSA Layer layer. The layer takes a softmax across queries, this forces the queries to compete for keys (pixel features). We see improvement in results by using this layer, over not using it (See Table 3).

Competition through Spatial Broadcast Decoder Finally, we use a slot wise reconstruction decoder, i.e., every slot is separately fed to the decoder for reconstruction (Figure 1). The final reconstruction is obtained by mixing all reconstructions with a softmax over the respective masks. This creates competition over pixels in the final reconstructed image. Due to the mixing of the reconstructions, the slots get reinforced for explaining different parts of the input. Thus, providing a push to the competitive nature of the slots. This has further been discussed in Engelcke et al. (2020).

In summary, MaxPool layers, SSA Layer layer and the decoder lead to competition and specialization amongst the slots and as a result they try to explain different parts of the input.

3.3 ADVANTAGES

The main advantage of our method over Slot Attention is, that it is non-iterative and therefore provides a better time and space complexity (see table 1).

Scalability The Slot Attention module is an **iterative refinement method** that needs several iterations to converge to a fixed point. As the complexity of the task increases, so does the number of required iterations. Chang et al. (2023), while addressing some of the training instabilities of Slot Attention, require even more iterations. In the case

Table 1: Runtime and Memory Complexities of the grouping module of Slot Attention vs. SAMP. S denotes the number of slots, D is the dimensionality of the slots and the number of channels (these are assumed to be equal for simplicity), H and W are the height and width of the image. While SAMP uses another few layers for the competition, Slot Attention performs several iterations of the iterative refinement procedure.

	Slot Attention	SAMP (ours)
Time (Training)	$\mathcal{O}(T \cdot S \cdot H \cdot W \cdot D)$	$\mathcal{O}((L_{\text{comp}} + S) \cdot H \cdot W \cdot D)$
Space (Training)	$\mathcal{O}(T \cdot S \cdot H \cdot W \cdot D)$	$\mathcal{O}((L_{\text{comp}} + S) \cdot H \cdot W \cdot D)$
Time (Test)	$\mathcal{O}(T \cdot S \cdot H \cdot W \cdot D)$	$\mathcal{O}((L_{\text{comp}} + S) \cdot H \cdot W \cdot D)$
Space (Test)	$\mathcal{O}(S \cdot H \cdot W \cdot D)$	$\mathcal{O}((L_{\text{comp}} + S) \cdot H \cdot W \cdot D)$

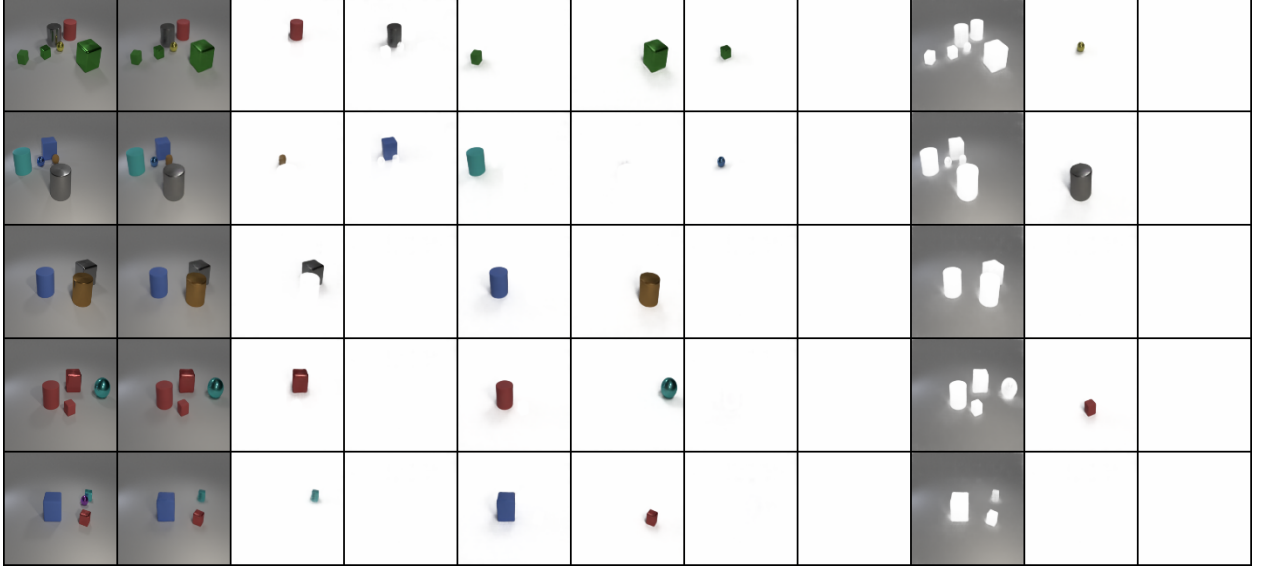


Figure 3: *Results on CLEVR6*: Visualized results of Multi-dSprites. The first column is the original image. The second column is the final reconstruction by the model, namely the weighted sum of individual reconstructions. Columns 3-11 are reconstructions of individual slots. The individual reconstructions are displayed without the mask.

of training on CLEVR, [Chang et al. \(2023\)](#) may utilize up to 11 iterations. These iterative methods, however, pose a significant challenge when it comes to scaling them for large-scale datasets.

SAMP distinguishes itself by not relying on recurrent neural networks (RNNs) and being a non-iterative method. This characteristic makes SAMP particularly well-suited for scaling up to handle large datasets and computational demands efficiently. SAMP’s non-iterative nature simplifies its application to extensive data processing and computation tasks, providing a distinct advantage over iterative methods like [Locatello et al. \(2020\)](#) and [Chang et al. \(2023\)](#).

4 EXPERIMENTS

Baselines We compare COP with the following commonly used slot extraction methods: Slot Attention ([Locatello et al., 2020](#)), IODINE ([Greff et al., 2019](#)) and MoNET ([Burgess et al., 2019](#)).

Datasets For the experiments we use three standard benchmarks common in prior work, namely CLEVR6, Multi-dSprites and Tetrominoes (introduced in [Kabra et al., 2019](#)). These are synthetic datasets which provide ground truth segmentation masks for evaluation. CLEVR consists of rendered scenes of 3D geometric shapes, e.g., spheres, cubes or cuboids, with a single light source. CLEVR6 is a subset of CLEVR which only contains a maximum of six objects. Multi-dSprites consists of 2-5 objects, which are 2D shapes (ellipses, hearts, squares) in different colors on black background. Tetrominoes consists of always three objects, namely 2D shapes from the tetris game.

Occlusion is an important property that largely defines the difficulty of a dataset. In tetrominoes there is no occlusion between objects. For CLEVR6, objects may partially occlude each other, though usually a single object is not occluded

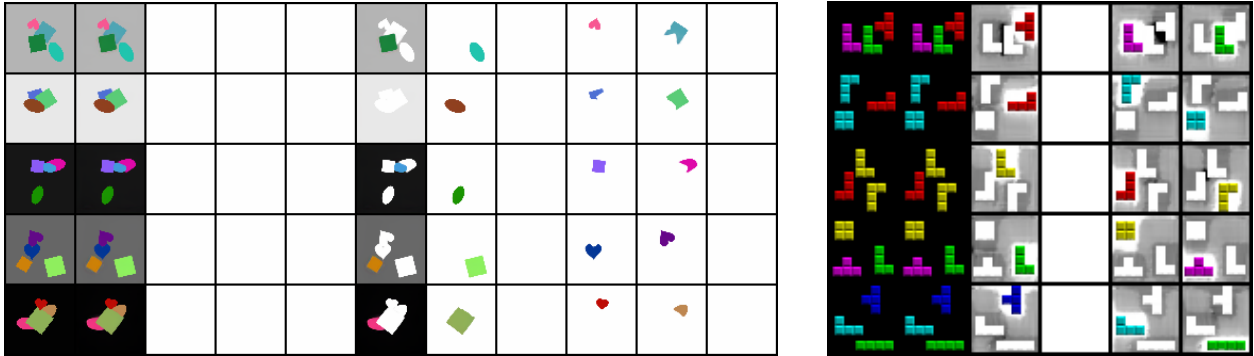


Figure 4: *Left*: Reconstructions of Multi-dSprites. The first column is the original image. The second column is the weighted sum of individual reconstructions where the weights come from the masks to which a pixel-wise softmax was applied. Columns 3-11 are reconstructions of individual slots. *Right*: Reconstructions of Tetrominoes. Again, the first column is the original image, while the second column is the weighted sum of individual reconstructions. Columns 3-6 are reconstructions of individual slots. The individual reconstructions are displayed without the mask.

by more than one other object. In Multi-dSprites an object may be occluded by several other objects, which makes it the most challenging of the three tasks in this regard.

For all these datasets we follow the protocol mentioned in Slot Attention (Locatello et al., 2020). CLEVR6 consists of 70k, and Multi-dSprites and Tetrominoes of 60k training samples. We partition the training set into a training and validation split to choose hyperparameters. For testing we trained on the whole training set and tested on 320 test samples (same as Slot Attention).

Training For the encoder and decoder architectures we follow the protocol described in (Locatello et al., 2020). For the grouping module we use the architecture described in section 3 with slight modifications for each dataset. First, we adjust the number of slots to equal or somewhat more than maximum number of objects of the according dataset. Since we have seen degrading performance of too many slots (also see section 5). Specifically, for CLEVR6 and Multi-dSprites we use nine slots, and for Tetrominoes we use four slots (three object slots and one background slots). All datasets are trained using a mean-squared-error reconstruction loss. Full details about the hyperparameters can be found in the appendix A.2.

Evaluation metric All of the above-mentioned datasets contain ground truth masks that enable evaluation of the image segmentations. We use the Adjusted Rand Index (Rand, 1971; Hubert & Arabie, 1985) as evaluation metric. This is a metric in the interval $[-0.5, 1.0]$ that compares two cluster assignments, whereas a score of 1 means the clusters are identical, a score of 0 would mean roughly equal to random assignment and -0.5 the worst assignment possible (worse than random). As common in the literature (Locatello et al., 2020; Engelcke et al., 2021) we omit the background ground truth masks in the evaluation, since we mostly want to distinguish between foreground objects. This metric is referred to as the FG-ARI.

Results SAMP is competitive for CLEVR6, but outperforms Slot Attention in Multi-dSprites and Tetrominoes. SAMP achieves state-of-the-art performance — on par with Slot Attention — when averaged over all three datasets. An overview of the results is given in table 2. Qualitatively we can see in figures 4 and 3 that the slot reconstructions are semantically meaningful and of high quality. Interestingly, some slots seem to specialize. E.g., for CLEVR6 and

Table 2: Results of different slot extraction methods on several datasets. Prior methods averaged over five seeds, while our results were averaged over three seeds.

Method	CLEVR6	Multi-dSprites	Tetrominoes	Average
Slot Attention	98.8 $\pm$ 0.3	91.3 $\pm$ 0.3	99.5 $\pm$ 0.2	96.53
IODINE	98.8 $\pm$ 0.0	76.7 $\pm$ 5.6	99.2 $\pm$ 0.4	91.57
MONET	96.2 $\pm$ 0.6	90.4 $\pm$ 0.8	n/a	93.30
SAMP (ours)	97.6 $\pm$ 0.6	92.3 $\pm$ 0.2	99.8 $\pm$ 0.1	96.57

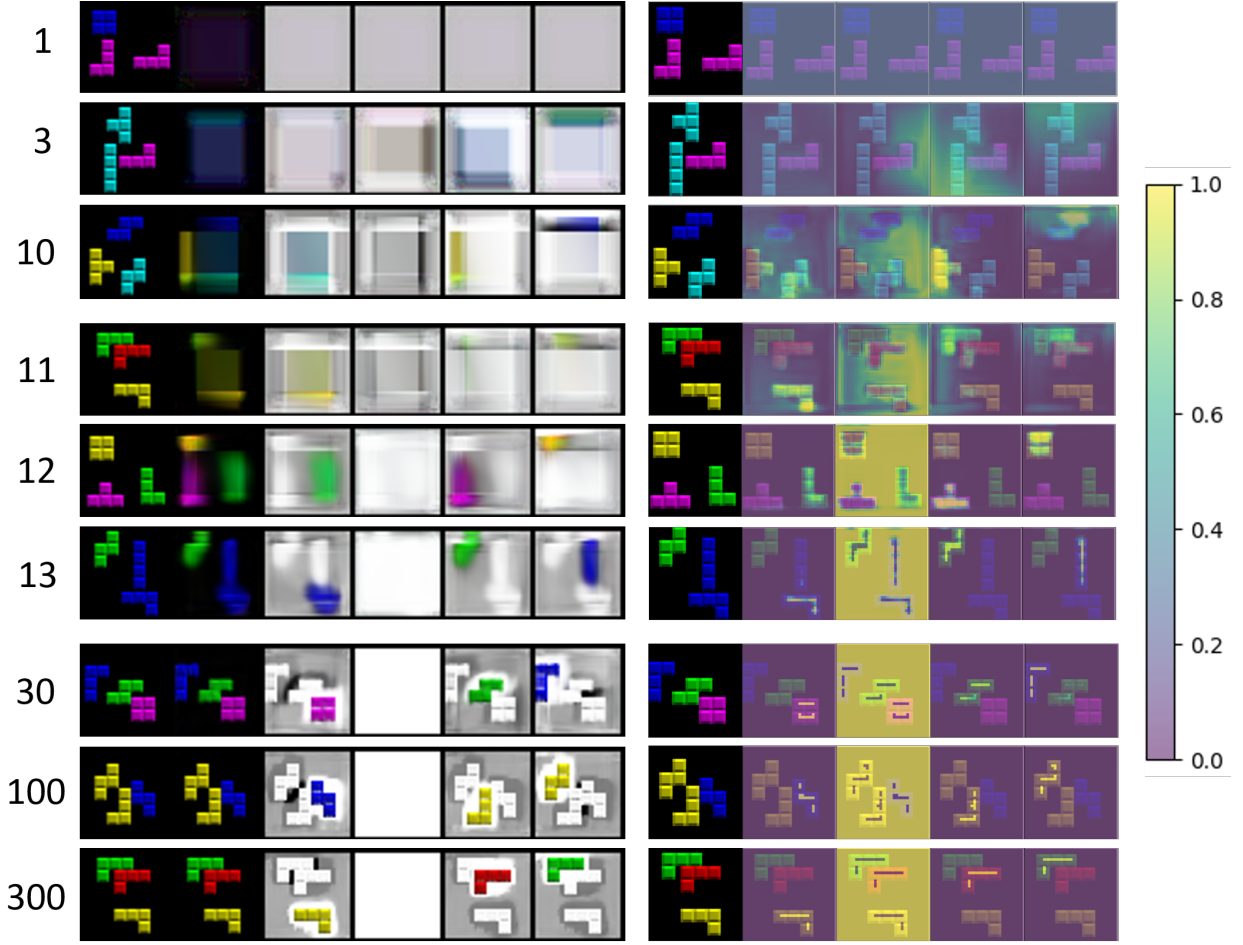


Figure 5: *Reconstructions and visualized attention heatmaps of slots over pixel features during training on Tetrominoes*: The numbers on the left denote the completed training epochs, the left group of images are reconstructions, whereas the right group are visualized attention maps. The columns of the reconstruction images are in the following order: (col. 1) ground truth image, (col. 2) final reconstruction (cols. 3-6) individual slot reconstructions. The columns of the visualized attention maps are in the following order: (col.1) ground truth image, (cols. 2-5) individual attention maps of the queries over the keys (i.e. the projected pixel features).

Tetrominoes there seems to be a clear specialization of the second slot to capture the gray background. This is also consistent with our observations of the slots over the training process. Non-background slots don’t seem to specialize in the position, shape, size nor color of the objects. Interestingly, some slots are not used in *any* of the reconstructions, which can be regarded as a specialization on ”not interfering” with other reconstructions.

5 ANALYSIS AND ABLATION STUDIES

Learned query visualization We visualized the attention of a slot over the keys of the pixel features. The results on the Tetrominoes dataset are in figure 5 (for further visualizations, please see appendix A.1). For the Tetrominoes run in figure 5, we can roughly identify three phases.

- **Warmup phase** Up until epoch 10, the reconstructions are poor and the attention maps are not sharp nor object-specific.
- **Transition phase** From epoch 11 to 13, there is a transition phase. While reconstructions are still poor after epoch 11, they have become sharper and object-specific after epoch 13. Also the attention maps clearly indicate that the queries are focusing on object-specific parts of the input.

- **Refinement phase** The objects at epoch 30 are still somewhat blurry, but are more detailed refined over the next several hundred epochs. For the attention maps, we can see that between epoch 13 and epoch 30 the attention maps change little. From epoch 30 onwards, the queries attend to very similar pixels of individual objects.

Which attention mechanism to use To understand the impact of the attention mechanism, we tried several ablations.

- **SAMP w/o SSA Layer** To check if SSA Layer over pixel features is beneficial at all, we omit the attention mechanism and directly reconstruct the images from the queries.
- **SAMP w/ Cross-Attention** Furthermore we substitute the SSA Layer with the common Cross-Attention (Vaswani et al., 2017).
- **SAMP w/ Attention as in Slot Attention** we substitute the SSA Layer with the attention variant used in Slot Attention (Locatello et al., 2020). Note that the GRU, MLPs at the end of each iteration were not used. Also we just applied the attention-variant once in a non-iterative fashion.

The results can be found in table 3. With four slots not using an attention-variant leads to a substantial drop in the FG-ARI score by 1.8. We conclude that competition of slots over pixel features is a useful bias for this architecture.

The effect of the maximum number of slots To gain further insights on how SAMP generalizes to more slots, we ran an ablation on Tetrominoes (Table 3). While SAMP manages to efficiently solve the task for six slots — which are 50% more than the ideal number slots — the performance largely degrades at nine or more slots. As visible in figure 8, oftentimes information about a single object is then shared among two slots.

Table 3: FG-ARI Mean and standard deviations over three seeds on the Tetrominoes dataset. *Left*: Ablations on the number of maximum slots. The ideal number of slots is four (i.e., three foreground objects and the background). If the number of slots is six (i.e., 50% higher than the ideal number of slots), the model still learns reasonably. If the number of slots becomes twice the number of ideal slots, the models performance degrades rapidly. *Right*: A comparison of attention-variants. The currently chosen attention-variant has the strongest performance. Using no attention mechanism (i.e., using the queries directly for reconstruction), Cross-Attention or the attention mechanism used in the Slot Attention module degrade performance.

Configuration	Tetrominoes	Attention variants (4 slots)	Tetrominoes
SAMP (4 slots)	99.77 $\pm$ 0.12	SAMP	99.77 $\pm$ 0.12
SAMP (6 slots)	99.58 $\pm$ 0.65	SAMP w/o Attention	97.97 $\pm$ 0.20
SAMP (8 slots)	89.13 $\pm$ 18.58	SAMP w/ Attn as in Slot Attention	91.07 $\pm$ 15.23
SAMP (9 slots)	84.21 $\pm$ 13.62	SAMP w/ Cross-Attention	90.92 $\pm$ 14.87
SAMP (16 slots)	63.99 $\pm$ 1.57		

6 LIMITATIONS AND DISCUSSION

Sensitivity to the maximum number of slots One down-side of SAMP is, that the number of objects is not adjustable during or after training. Other methods also suffer from a sensitivity to the number of slots during training. This problem can be improved by training with more slots and flexibly adjust the number of slots during inference (Zimmermann et al., 2023).

Connections to continual learning The ability to learn abstract, object-centric representations in an unsupervised manner is a key challenge for continual learning agents that must operate in dynamic, continually changing environments (Greff et al., 2020; Wang et al., 2024). By extracting slots that correspond to object concepts from raw sensory data, SAMP provides a mechanism for continually building up a structured understanding of the world in a continual learning setting. As an agent experiences more data over time, the slots can be refined and updated to reflect new objects and concepts encountered (e.g. similar to Seitzer et al. (2023)). This allows for efficient knowledge accumulation and transfer as the agent faces distribution shifts and must adapt to new situations, a core goal of continual learning. Furthermore, our simple, scalable architecture based on standard primitives like convolutions and attention makes SAMP well-suited for continual learning scenarios with constrained computational resources. Overall, unsupervised object-centric representation learning enabled by approaches like SAMP can serve as a fundamental building block for continual learning agents.

7 CONCLUSION

In this paper we presented a novel method for object-centric learning that is based on competition, called Simplified Slot Attention with Max Pool Priors (SAMP). SAMP is simple, scalable, fully-differentiable and contrary to the widely applied Slot Attention, it is non-iterative. It comes with the down-side of not being able to adjust to a different number of slots between training and test time. Empirically our method is competitive or outperforms existing slot extraction methods.

ACKNOWLEDGEMENTS

The ELLIS Unit Linz, the LIT AI Lab, and the Institute for Machine Learning are supported by the Federal State of Upper Austria. We thank the projects Medical Cognitive Computing Center (MC3), INCONTROL-RL (FFG-881064), PRIMAL (FFG-873979), S3AI (FFG-872172), EPILEPSIA (FFG-892171), AIRI FG 9-N (FWF-36284, FWF-36235), AI4GreenHeatingGrids (FFG- 899943), INTEGRATE (FFG-892418), ELISE (H2020-ICT-2019-3 ID: 951847), Stars4Waters (HORIZON-CL6-2021-CLIMATE-01-01). We thank the European High Performance Computing initiative for providing computational resources (EHPC-DEV-2023D08-019). We thank Audi.JKU Deep Learning Center, TGW LOGISTICS GROUP GMBH, Silicon Austria Labs (SAL), FILL Gesellschaft mbH, Anyline GmbH, Google, ZF Friedrichshafen AG, Robert Bosch GmbH, UCB Biopharma SRL, Merck Healthcare KGaA, Verbund AG, Software Competence Center Hagenberg GmbH, Borealis AG, TÜV Austria, Frauscher Sensonic, TRUMPF, and the NVIDIA Corporation.

REFERENCES

- Per Andersen, G. N. Gross, Torkel Lomo, and Ole Sveen. Participation of inhibitory and excitatory interneurons in the control of hippocampal cortical output. *UCLA Forum Med Sci*, 11:415–465, 1969.
- Christopher P. Burgess, Loic Matthey, Nicholas Watters, Rishabh Kabra, Irina Higgins, Matt Botvinick, and Alexander Lerchner. Monet: Unsupervised scene decomposition and representation, 2019.
- Michael Chang, Thomas L. Griffiths, and Sergey Levine. Object representations as fixed points: Training iterative refinement algorithms with implicit differentiation, 2023.
- Junyoung Chung, Çağlar Gülçehre, KyungHyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. *CoRR*, abs/1412.3555, 2014. URL <http://arxiv.org/abs/1412.3555>.
- D. Comaniciu and P. Meer. Mean shift: a robust approach toward feature space analysis. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 24(5):603–619, 2002. doi: 10.1109/34.1000236.
- Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. pp. 248–255, 2009. doi: 10.1109/CVPR.2009.5206848.
- John C. Eccles, Masao Ito, and János Szentágothai. *The Cerebellum as a Neuronal Machine*. 1967.
- Stephen A. Ellias and Stephen Grossberg. Pattern formation, contrast control, and oscillations in the short term memory of shunting on-center off-surround networks. *Biol. Cybernetics*, 20:69–98, 1975. doi: 10.1007/BF00327046. URL <https://doi.org/10.1007/BF00327046>.
- Martin Engelcke, Adam R. Kosiorek, Oiwi Parker Jones, and Ingmar Posner. GENESIS: generative scene inference and sampling with object-centric latent representations. *CoRR*, abs/1907.13052, 2019. URL <http://arxiv.org/abs/1907.13052>.
- Martin Engelcke, Oiwi Parker Jones, and Ingmar Posner. Reconstruction bottlenecks in object-centric generative models, 2020.
- Martin Engelcke, Oiwi Parker Jones, and Ingmar Posner. GENESIS-V2: inferring unordered object representations without iterative refinement. *CoRR*, abs/2104.09958, 2021. URL <https://arxiv.org/abs/2104.09958>.
- Bard Ermentrout. Complex dynamics in winner-take-all neural nets with slow inhibition. *Neural Networks*, 5(3): 415–431, 1992. ISSN 0893-6080. doi: [https://doi.org/10.1016/0893-6080\(92\)90004-3](https://doi.org/10.1016/0893-6080(92)90004-3). URL <https://www.sciencedirect.com/science/article/pii/0893608092900043>.

- Ning Gao, Bernard Hohmann, and Gerhard Neumann. Enhancing interpretable object abstraction via clustering-based slot initialization, 2023.
- R. Givan, T. Dean, and M. Greig. Equivalence notions and model minimization in Markov decision processes. *Artificial Intelligence*, 147(1):163–223, 2003. doi: 10.1016/S0004-3702(02)00376-4.
- Klaus Greff, Raphaël Lopez Kaufman, Rishabh Kabra, Nick Watters, Chris Burgess, Daniel Zoran, Loïc Matthey, Matthew M. Botvinick, and Alexander Lerchner. Multi-object representation learning with iterative variational inference. *CoRR*, abs/1903.00450, 2019. URL <http://arxiv.org/abs/1903.00450>.
- Klaus Greff, Sjoerd van Steenkiste, and Jürgen Schmidhuber. On the binding problem in artificial neural networks. *CoRR*, abs/2012.05208, 2020. URL <https://arxiv.org/abs/2012.05208>.
- K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Spatial pyramid pooling in deep convolutional networks for visual recognition. *CoRR*, abs/1406.4729, 2014. URL <http://arxiv.org/abs/1406.4729>.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *CoRR*, abs/1512.03385, 2015. URL <http://arxiv.org/abs/1512.03385>.
- Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross B. Girshick. Mask R-CNN. *CoRR*, abs/1703.06870, 2017. URL <http://arxiv.org/abs/1703.06870>.
- Lawrence Hubert and Phipps Arabie. Comparing partitions. *Journal of Classification*, 2(1):193–218, 1985.
- Baoxiong Jia, Yu Liu, and Siyuan Huang. Improving object-centric learning with query optimization, 2023.
- Rishabh Kabra, Chris Burgess, Loic Matthey, Raphael Lopez Kaufman, Klaus Greff, Malcolm Reynolds, and Alexander Lerchner. Multi-object datasets. <https://github.com/deepmind/multi-object-datasets/>, 2019.
- Jinwoo Kim, Janghyuk Choi, Ho-Jin Choi, and Seon Joo Kim. Shepherding slots to objects: Towards stable and robust object-centric learning, 2023.
- A. Krizhevsky, I. Sutskever, and G. E. Hinton. ImageNet classification with deep convolutional neural networks. In F. Pereira, C. J. C. Burges, L. Bottou, and K. Q. Weinberger (eds.), *Advances in Neural Information Processing Systems 25*, pp. 1097–1105. Curran Associates, Inc., 2012.
- T. D. Kulkarni, K. Narasimhan, A. Saeedi, and Josh J. Tenenbaum. Hierarchical deep reinforcement learning: Integrating temporal abstraction and intrinsic motivation. In D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett (eds.), *Advances in Neural Information Processing Systems*, volume 29, pp. 3675–3683. Curran Associates, Inc., 2016.
- Y. LeCun, F.-J. Huang, and L. Bottou. Learning methods for generic object recognition with invariance to pose and lighting. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE Press, 2004.
- Daniel K. Lee, Laurent Itti, Christof Koch, and Jochen Braun. Attention activates winner-take-all competition among visual filters. *Nat Neurosci*, 2(4):375–381, 1999. doi: 10.1038/7286.
- L. Li, T. J. Walsh, and M. L. Littman. Towards a unified theory of state abstraction for MDPs. In *Ninth International Symposium on Artificial Intelligence and Mathematics (ISAIM)*, 2006.
- Zhixuan Lin, Yi-Fu Wu, Skand Vishwanath Peri, Weihao Sun, Gautam Singh, Fei Deng, Jindong Jiang, and Sungjin Ahn. SPACE: unsupervised object-oriented scene representation via spatial attention and decomposition. *CoRR*, abs/2001.02407, 2020. URL <http://arxiv.org/abs/2001.02407>.
- R. Lippmann. An introduction to computing with neural nets. *IEEE ASSP Magazine*, 4(2):4–22, 1987. doi: 10.1109/MASSP.1987.1165576.
- S. Lloyd. Least squares quantization in pcm. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982. doi: 10.1109/TIT.1982.1056489.

- Francesco Locatello, Dirk Weissenborn, Thomas Unterthiner, Aravindh Mahendran, Georg Heigold, Jakob Uszkoreit, Alexey Dosovitskiy, and Thomas Kipf. Object-centric learning with slot attention. *CoRR*, abs/2006.15055, 2020. URL <https://arxiv.org/abs/2006.15055>.
- Wolfgang Maass. Neural computation with winner-take-all as the only nonlinear operation. 12, 1999. URL https://proceedings.neurips.cc/paper_files/paper/1999/file/1c54985e4f95b7819ca0357c0cb9a09f-Paper.pdf.
- Wolfgang Maass. On the computational power of winner-take-all. *Neural Comput.*, 12(11):2519–2535, nov 2000. ISSN 0899-7667. doi: 10.1162/089976600300014827. URL <https://doi.org/10.1162/089976600300014827>.
- V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, , and D. Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. doi: 10.1038/nature14236.
- F. Paischer, T. Adler, V. Patil, A. Bitto-Nemling, M. Holzleitner, S. Lehner, H. Eghbal-Zadeh, and S. Hochreiter. History compression via language models in reinforcement learning. In K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu, and S. Sabato (eds.), *Proceedings of the 39th International Conference on Machine Learning*, pp. 17156–17185, 2022.
- Vihang Patil, Markus Hofmarcher, Marius-Constantin Dinu, Matthias Dorfer, Patrick M Blies, Johannes Brandstetter, José Arjona-Medina, and Sepp Hochreiter. Align-RUDDER: Learning from few demonstrations by reward redistribution. In Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvari, Gang Niu, and Sivan Sabato (eds.), *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pp. 17531–17572. PMLR, 17–23 Jul 2022. URL <https://proceedings.mlr.press/v162/patil22a.html>.
- Vihang Patil, Markus Hofmarcher, Elisabeth Rumetshofer, and Sepp Hochreiter. Contrastive abstraction for reinforcement learning. In *NeurIPS 2023 Workshop on Generalization in Planning*, 2023. URL <https://openreview.net/forum?id=oMkUQKfsCU>.
- Adeel Pervez, Phillip Lippe, and Efstratios Gavves. Differentiable mathematical programming for object-centric representation learning, 2022.
- Aditya Ramesh, Mikhail Pavlov, Gabriel Goh, Scott Gray, Chelsea Voss, Alec Radford, Mark Chen, and Ilya Sutskever. Zero-shot text-to-image generation. *CoRR*, abs/2102.12092, 2021. URL <https://arxiv.org/abs/2102.12092>.
- H. Ramsauer, B. Schäfl, J. Lehner, P. Seidl, M. Widrich, L. Gruber, M. Holzleitner, M. Pavlović, G. K. Sandve, V. Greiff, D. Kreil, M. Kopp, G. Klambauer, J. Brandstetter, and S. Hochreiter. Hopfield networks is all you need. *ArXiv*, 2008.02217, 2020.
- William M. Rand. Objective criteria for the evaluation of clustering methods. *Journal of the American Statistical Association*, 66(336):846–850, 1971. ISSN 01621459. URL <http://www.jstor.org/stable/2284239>.
- B. Ravindran and A. G. Barto. SMDP homomorphisms: An algebraic approach to abstraction in semi-Markov decision processes. In *Proc. of the 18th Int. Joint Conf. on Artificial Intelligence (IJCAI’03)*, pp. 1011–1016, San Francisco, CA, USA, 2003. Morgan Kaufmann Publishers Inc.
- J. Schmidhuber. Deep learning in neural networks: An overview. *Neural Networks*, 61:85–117, 2015. doi: 10.1016/j.neunet.2014.09.003.
- Maximilian Seitzer, Max Horn, Andrii Zadaianchuk, Dominik Zietlow, Tianjun Xiao, Carl-Johann Simon-Gabriel, Tong He, Zheng Zhang, Bernhard Schölkopf, Thomas Brox, and Francesco Locatello. Bridging the gap to real-world object-centric learning, 2023.
- D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. P. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016. doi: 10.1038/nature16961.

- Gautam Singh, Fei Deng, and Sungjin Ahn. Illiterate dall-e learns to compose, 2022.
- Elizabeth S. Spelke and Katherine D. Kinzler. Core knowledge. *Developmental Science*, 10(1):89–96, 2007.
- N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *J. Mach. Learn. Res.*, 15:1929–1958, 2014a.
- Rupesh K Srivastava, Jonathan Masci, Sohrob Kazerounian, Faustino Gomez, and Jürgen Schmidhuber. Compete to compute. In C.J. Burges, L. Bottou, M. Welling, Z. Ghahramani, and K.Q. Weinberger (eds.), *Advances in Neural Information Processing Systems*, volume 26. Curran Associates, Inc., 2013. URL https://proceedings.neurips.cc/paper_files/paper/2013/file/8f1d43620bc6bb580df6e80b0dc05c48-Paper.pdf.
- Rupesh Kumar Srivastava, Jonathan Masci, Faustino Gomez, and Jürgen Schmidhuber. Understanding locally competitive networks, 2014b.
- R. S. Sutton, D. Precup, and S. P. Singh. Between MDPs and Semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1-2):181–211, 1999.
- Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott E. Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. *CoRR*, abs/1409.4842, 2014. URL <http://arxiv.org/abs/1409.4842>.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *CoRR*, abs/1706.03762, 2017. URL <http://arxiv.org/abs/1706.03762>.
- A. S. Vezhnevets, S. Osindero, T. Schaul, N. Heess, M. Jaderberg, D. Silver, and K. Kavukcuoglu. FeUdal networks for hierarchical reinforcement learning. *arXiv*, abs/1703.01161, 2017.
- Dong Wang and Xiaoyang Tan. Unsupervised feature learning with c-svddnet, 2014.
- Liyuan Wang, Xingxing Zhang, Hang Su, and Jun Zhu. A comprehensive survey of continual learning: Theory, method and application, 2024.
- Nicholas Watters, Loic Matthey, Christopher P. Burgess, and Alexander Lerchner. Spatial broadcast decoder: A simple architecture for learning disentangled representations in vaes, 2019.
- M. Widrich, M. Hofmarcher, V. P. Patil, A. Bitto-Nemling, and S. Hochreiter. Modern Hopfield networks for return decomposition for delayed rewards. In *Deep RL Workshop at NeurIPS 2021*, 2021. URL <https://openreview.net/forum?id=t0PQSDcqAiy>.
- Ziyi Wu, Nikita Dvornik, Klaus Greff, Thomas Kipf, and Animesh Garg. Slotformer: Unsupervised visual dynamics simulation with object-centric models, 2023.
- Roland S. Zimmermann, Sjoerd van Steenkiste, Mehdi S. M. Sajjadi, Thomas Kipf, and Klaus Greff. Sensitivity of slot-based object-centric models to their number of slots, 2023.

A APPENDIX

A.1 ABLATIONS

In this section we add additional figures for the ablation studies. The figures are taken from **the best prediction of the worst run** among all the seeds. Figures 6, 7 and 8 are reconstructions of the respective architectures.

A.2 HYPERPARAMETERS

A.2.1 ENCODER ARCHITECTURES

For the encoder architectures we follow the protocol given in Locatello et al. (2020). The detailed layer descriptions for CLEVR6 can be found in table 4. Similarly the encoder layer descriptions for Multi-dSprites and Tetrominoes are explained in table 5.

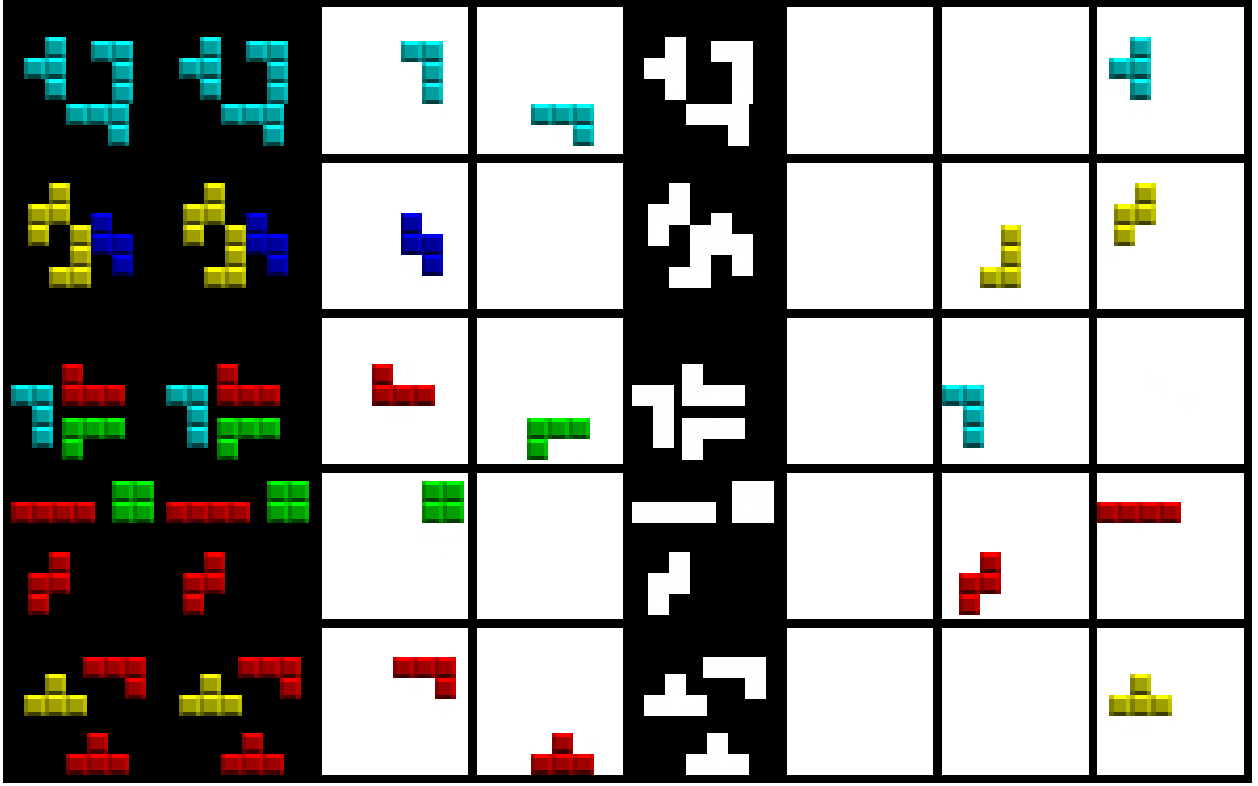


Figure 6: Ablation of Tetrominoes with six slots: The reconstructions still work reasonably well.

Table 4: CNN encoder for CLEVR6.

Type	Size/Channels	Activation	Comment
Conv 5×5	64	ReLU	Stride: 1
Conv 5×5	64	ReLU	Stride: 1
Conv 5×5	64	ReLU	Stride: 1
Conv 5×5	64	ReLU	Stride: 1
Position Embedding	-	-	See Section A.2.4
Flatten	axis: $[0, 1 \times 2, 3]$	-	Flatten x, y pos.
Layer Norm	-	-	-
MLP (per location)	64	ReLU	-
MLP (per location)	64	-	-

A.2.2 COMPETITION ENCODER ARCHITECTURES

The layer descriptions for the competition encoders can be found in tables 6, 8 and 7. Note that before the Competition encoder, a positional bias is added to the pixel features, as described in appendix A.2.4.

A.2.3 DECODER ARCHITECTURES

Again, we follow the protocol of Locatello et al. (2020). Note that before the decoding, a positional bias is added to the spatial map, as described in appendix A.2.4. The detailed layer descriptions for CLEVR6 can be found in table 9. Similarly the encoder layer descriptions for Multi-dSprites and Tetrominoes are explained in table 10.

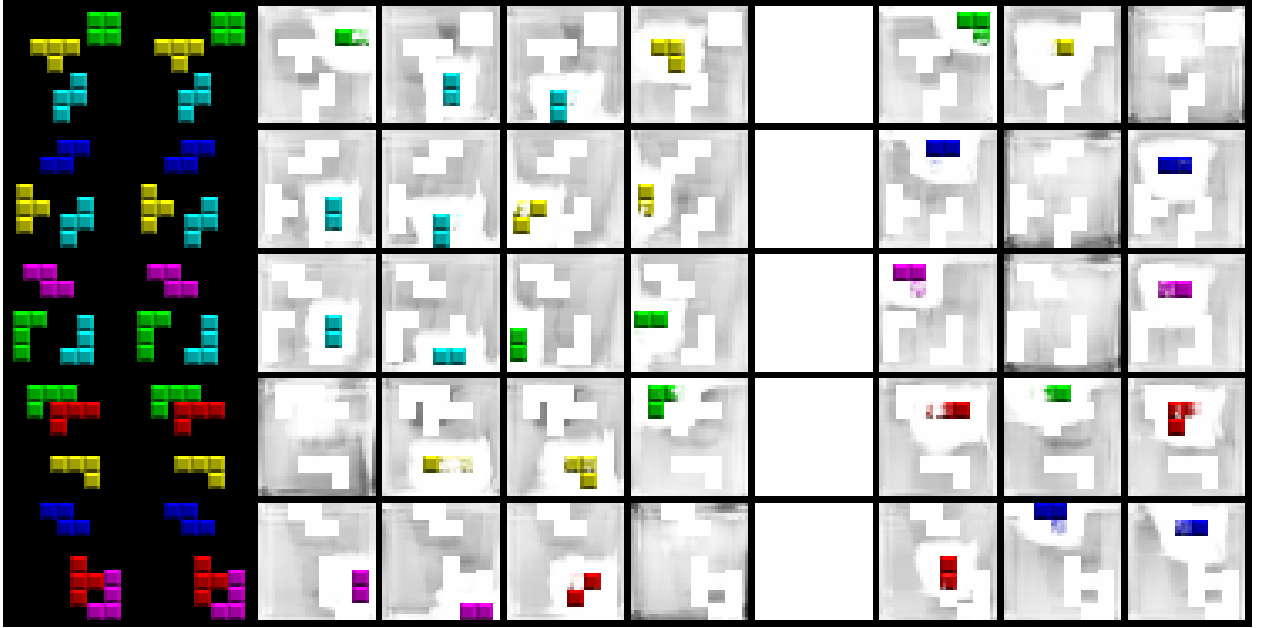


Figure 7: *Ablation of Tetrominoes with 8 slots*: Since there are twice as many slots than the ideal four for Tetrominoes (i.e., three objects, one background), a single object is often represented in two slots.

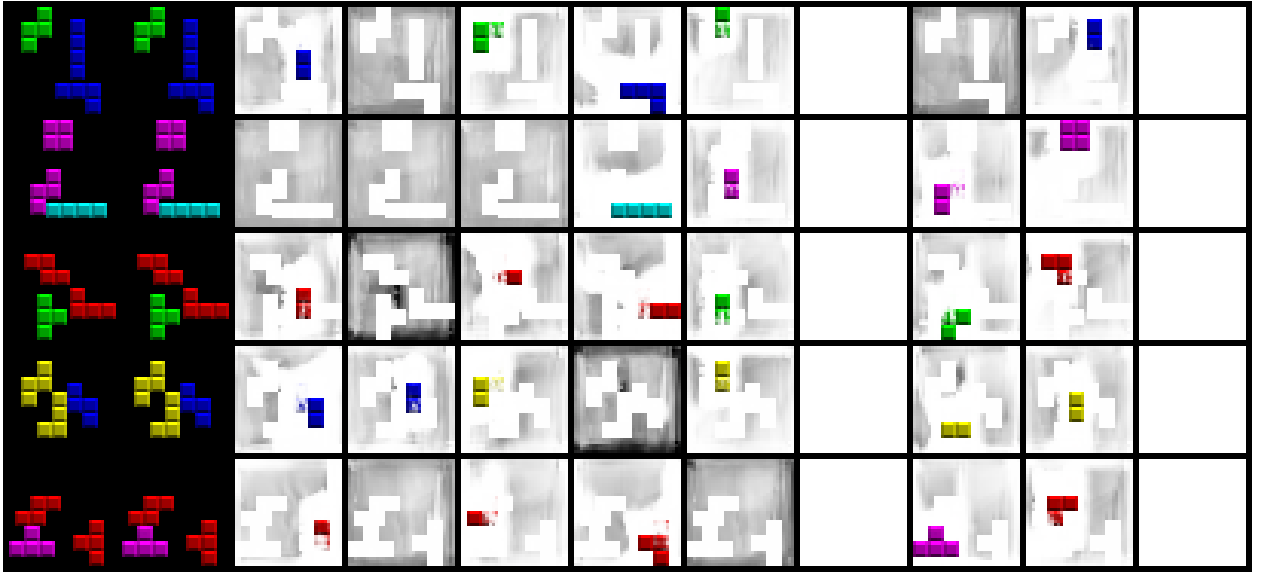


Figure 8: *Ablation of Tetrominoes with nine slots*: Since there are more than twice as many slots than the ideal four for Tetrominoes (i.e., three objects, one background), a single object is often represented in two slots.

A.2.4 POSITIONAL EMBEDDING

We follow the positional embedding given in [Locatello et al. \(2020\)](#). Summarized, for each pixel a 4-dimensional positional embedding is initialized that encodes the distance to all colors (normalized to the range $[0, 1]$). These 4 dimensional embeddings are then projected with a learnable matrix to the dimensionality of the pixel features.

Table 5: CNN encoder for Multi-dSprites and Tetrominoes.

Type	Size/Channels	Activation	Comment
Conv 5×5	32	ReLU	Stride: 1
Conv 5×5	32	ReLU	Stride: 1
Conv 5×5	32	ReLU	Stride: 1
Conv 5×5	32	ReLU	Stride: 1
Position Embedding	-	-	See Section A.2.4
Flatten	axis: $[0, 1 \times 2, 3]$	-	Flatten x, y pos.
Layer Norm	-	-	-
MLP (per location)	32	ReLU	-
MLP (per location)	32	-	-

Table 6: Competition encoder layers for Tetrominoes.

Type	Spatial Resolution	Size/Channels	Activation	Stride	Padding
Conv 5×5	35x35	32	LeakyReLU	1	2
MaxPool	17x17	32		2	0
Conv 5×5	17x17	32	LeakyReLU	1	2
MaxPool	8x8	32		2	0
Conv 5×5	8x8	32	LeakyReLU	1	2
MaxPool	4x4	32		2	0
Conv 5×5	4x4	32	LeakyReLU	1	2
MaxPool	2x2	32		2	0
Flatten	$2 \times 2 \rightarrow 4$	axis: $[0, 1 \times 2, 3]$			
FC	4	32	LeakyReLU		
FC	4	32			

A.3 LEARNED QUERY VISUALIZATIONS

Similarly to the Tetrominoes, we see that the recognition of objects gradually evolves for CLEVR6 and Multi-dSprites. The learning process over time of CLEVR6 is depicted in figure 9 for the reconstructions and in figure 10 for the attention heatmaps.

The learning process over time of Multi-dSprites is depicted in figure 11 for the reconstructions and figure 12 for the attention heatmaps. Although some slots do not show a visible attention map, we checked their numerical value and conclude that the attention values are chosen large enough to reconstruct the objects.

Table 7: Competition encoder layers for Multi-dSprites.

Type	Spatial Resolution	Size/Channels	Activation	Stride	Padding
Conv 5×5	64x64	64	LeakyReLU	1	2
MaxPool	64x64	64		2	0
Conv 5×5	32x32	64	LeakyReLU	1	2
MaxPool	32x32	64		2	0
Conv 5×5	16x16	64	LeakyReLU	1	2
MaxPool	16x16	64		2	0
Conv 5×5	8x8	64	LeakyReLU	1	2
MaxPool	8x8	64		2	0
Conv 5×5	4x4	64	LeakyReLU	1	2
MaxPool	4x4	64		1	0
Flatten	$3 \times 3 \rightarrow 9$	axis: $[0, 1 \times 2, 3]$			
FC	9	64	LeakyReLU		
FC	9	64			

Table 8: Competition encoder layers for CLEVR6.

Type	Spatial Resolution	Size/Channels	Activation	Stride	Padding
Conv 5×5	128x128	64	LeakyReLU	1	2
MaxPool	128x128	64		2	0
Conv 5×5	64x64	64	LeakyReLU	1	2
MaxPool	64x64	64		2	0
Conv 5×5	32x32	64	LeakyReLU	1	2
MaxPool	32x32	64		2	0
Conv 5×5	16x16	64	LeakyReLU	1	2
MaxPool	16x16	64		2	0
Conv 5×5	8x8	64	LeakyReLU	1	2
MaxPool	8x8	64		2	0
Conv 5×5	4x4	64	LeakyReLU	1	2
MaxPool	4x4	64		1	0
Flatten	$3 \times 3 \rightarrow 9$	axis: $[0, 1 \times 2, 3]$			
FC	9	64	LeakyReLU		
FC	9	64			

Table 9: Decoder layer description for CLEVR6.

Type	Spatial res.	Size/Channels	Activation	Comment
Spatial Broadcast	1	64	-	-
Position Embedding	8x8	64	-	See A.2.4
Conv 5×5	8x8	64	ReLU	Stride: 2
Conv 5×5	16x16	64	ReLU	Stride: 2
Conv 5×5	32x32	64	ReLU	Stride: 2
Conv 5×5	64x64	64	ReLU	Stride: 2
Conv 5×5	128x128	64	ReLU	Stride: 1
Conv 3×3	128x128	4	-	Stride: 1
Split Channels	128x128	RGB (3), mask (1)	Softmax on masks	-
Recombine Slots	128x128	-	-	-

Table 10: Decoder layer description for Multi-dSprites and Tetrominoes.

Type	Spatial res.	Size/Channels	Activation	Comment
Spatial Broadcast	1	32	-	-
Position Embedding	WxH	32	-	See A.2.4
Conv 5 x 5	WxH	32	ReLU	Stride: 1
Conv 5 x 5	WxH	32	ReLU	Stride: 1
Conv 5 x 5	WxH	32	ReLU	Stride: 1
Conv 3 x 3	WxH	4	-	Stride: 1
Split Channels	WxH	RGB (3), mask (1)	Softmax on masks	-
Recombine Slots	WxH	-	-	-

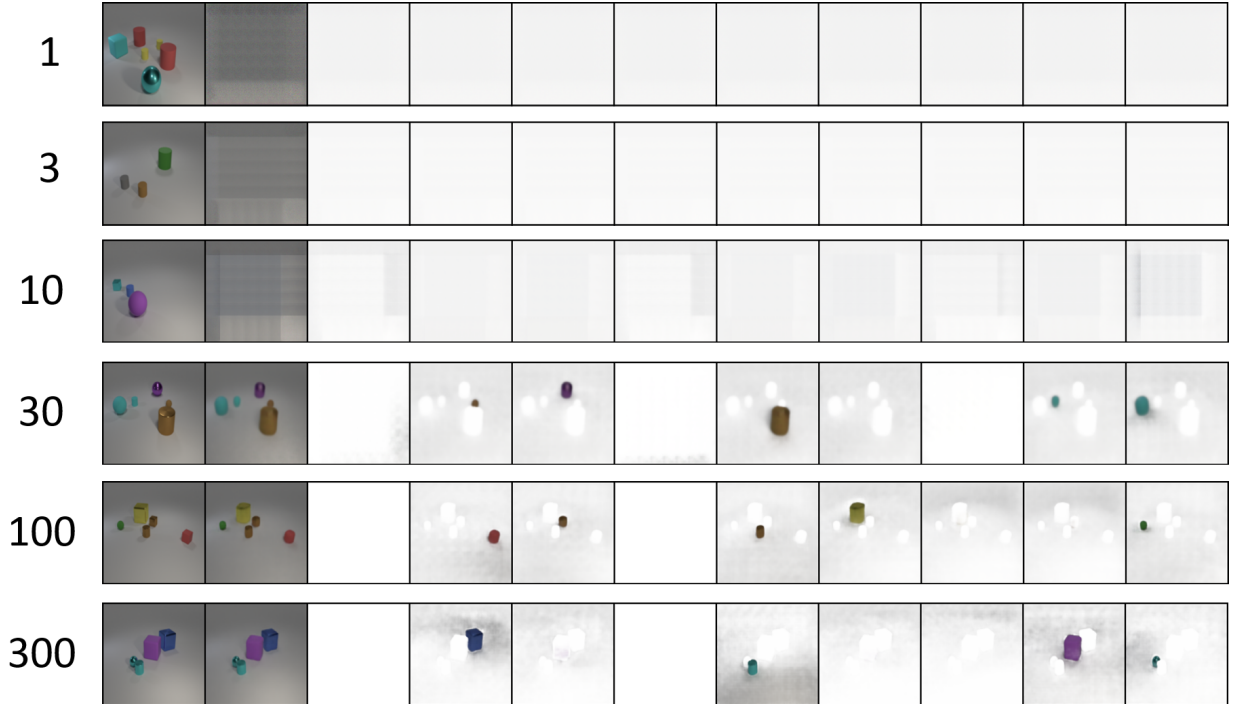


Figure 9: *Reconstructions over training of CLEVR6*: The numbers on the left denote the completed training epochs, whereas the images are reconstructions. The columns of the reconstruction images are in the following order: (col. 1) ground truth image, (col. 2) reconstruction (cols. 3-11) individual slot reconstructions.

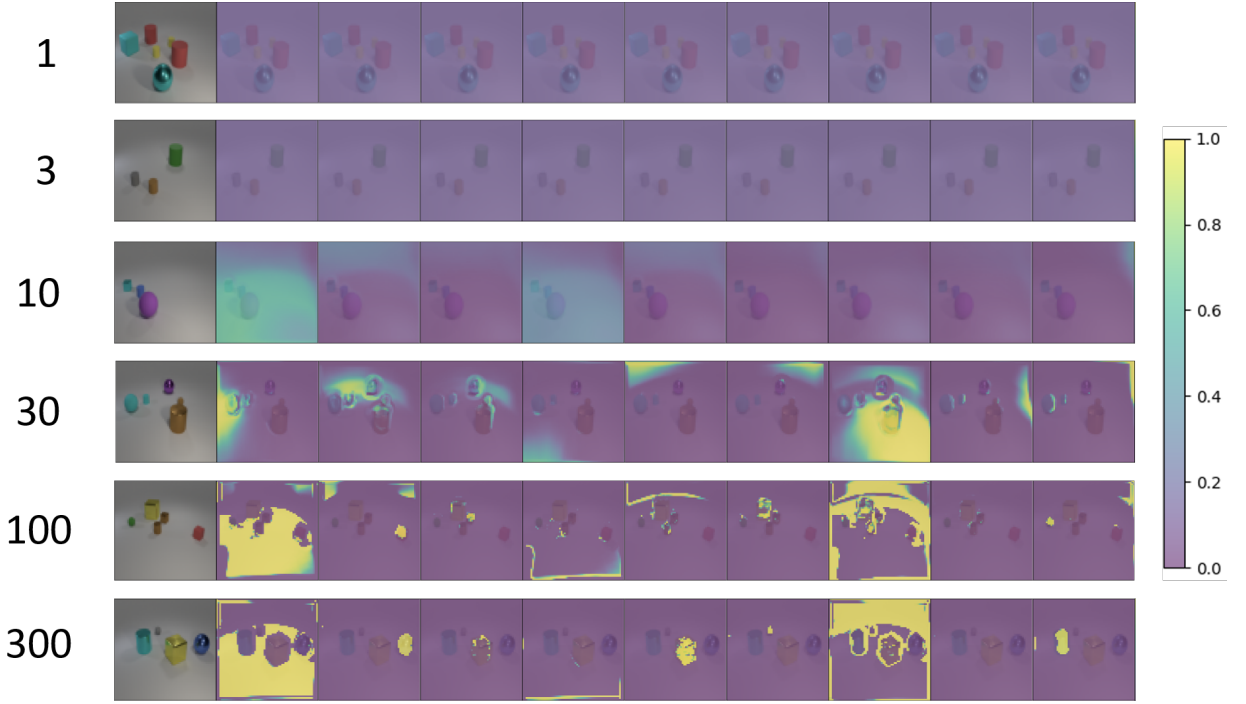


Figure 10: *Attention heatmaps during training of CLEVR6*: The numbers on the left denote the completed training epochs, whereas the images are visualized attention maps. The columns of the visualized attention maps are in the following order: (col.1) ground truth image, (cols. 2-10) individual attention maps of the queries over the keys (transformed pixel features).

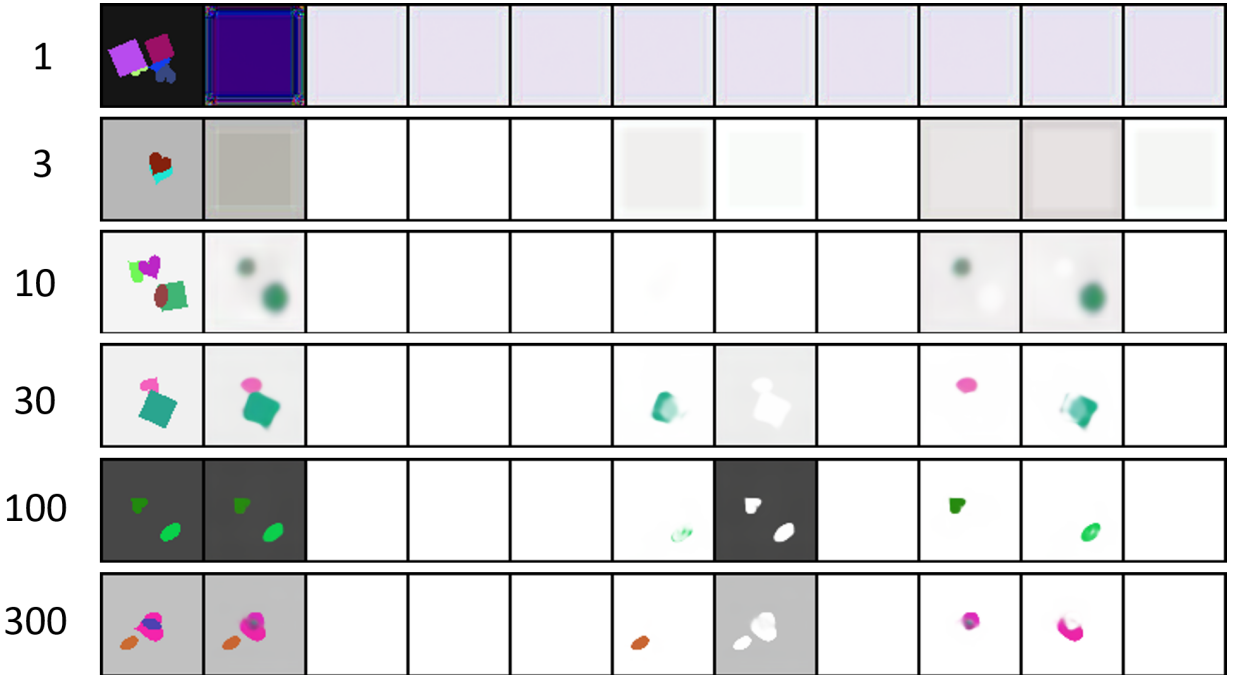


Figure 11: *Reconstructions over training of Multi-dSprites*: The numbers on the left denote the completed training epochs, whereas the images are reconstructions. The columns of the reconstruction images are in the following order: (col. 1) ground truth image, (col. 2) reconstruction (cols. 3-11) individual slot reconstructions.

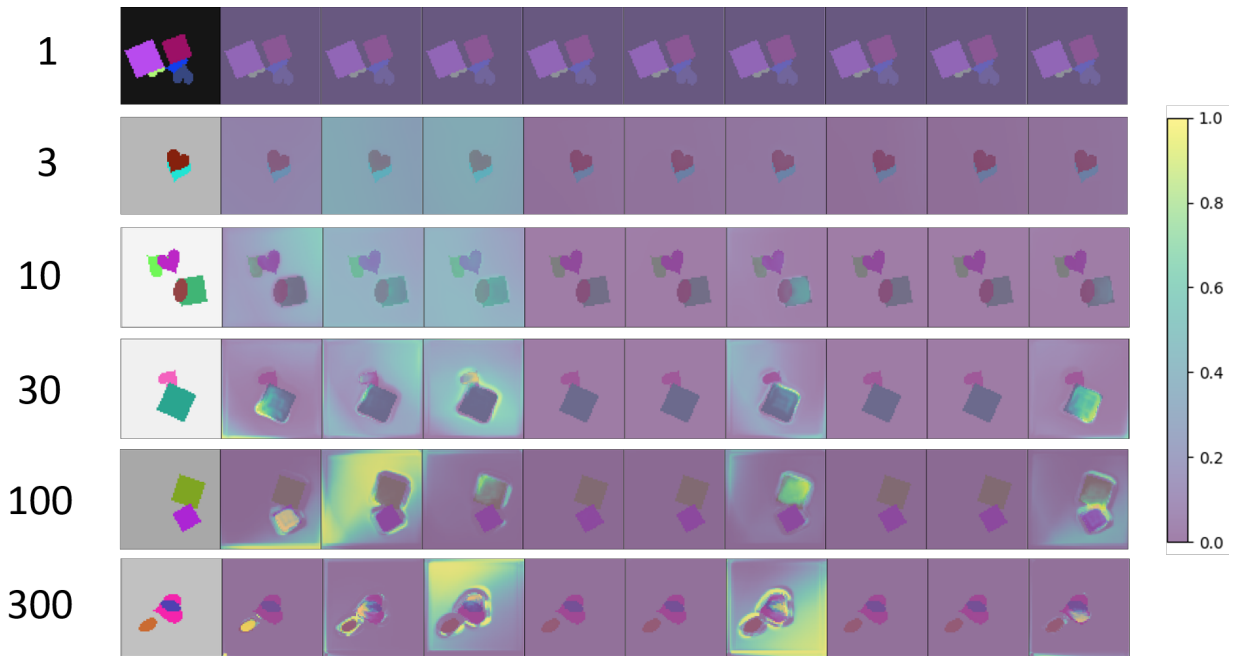


Figure 12: *Heatmaps of attention over training of Multi-dSprites*: The numbers on the left denote the completed training epochs, whereas the images are visualized attention maps. The columns of the visualized attention maps are in the following order: (col.1) ground truth image, (cols. 2-10) individual attention maps of the queries over the keys (transformed pixel features).